Create

Use a local theme when you need structural changes, not just color and spacing. Common reasons include custom page chrome, extra client-side behavior, branded metadata, and notebook wrappers that affect how the Typst source is final-formatted.

You can start small and expand only where needed.

Create a local theme

Built-in themes are compiled into the Calepin binary, so you customize them by copying one into your project first. Use `--theme` to choose which built-in theme gets ejected: that theme becomes the base for your local customizations.

```
calepin new theme                          # eject the default `calepin` theme to calepin_theme/
calepin new theme --theme calepin          # same, explicitly
calepin new theme --theme academic          # eject the `academic` theme to calepin_theme/
calepin new theme --theme calepin themes/my  # copy into a custom directory
```

Then point Calepin at your copy:

```
theme = "calepin_theme"
theme = "themes/my-theme" # with the custom name
```

Once copied, the theme is project-owned: edit its templates, styles, scripts, and `theme.toml` freely and keep it in version control.

Start small

A local theme can be tiny. You only need to include files you want to customize. Everything else falls back to the built-in `calepin` theme, so valid overrides include:

- only `layouts/webpage.html`
- only `layouts/notebook.html`
- only `notebook.typ.jinja`

Supporting assets (`partials`, `styles`, `scripts`) come from the selected theme and from any shared imports declared in `theme.toml`.

What can be customized

Use this map when you want to choose the right file first:

```
themes/my-theme/
  theme.toml            # theme metadata and shared imports
  layouts/
    webpage.html         # website page wrapper
    notebook.html        # standalone notebook HTML wrapper
    landing.html         # optional page-specific override
  partials/
    ...                  # reusable template fragments
  css/
    ...                  # theme CSS
  js/
    ...                  # theme JavaScript
  notebook.typ.jinja   # Typst template around notebook source
```

HTML templates

For a single HTML notebook, use `layouts/notebook.html`. For websites, the default entry is `layouts/webpage.html`.

Layouts are MiniJinja templates. The most common values are:

- `doc.head`, `doc.body_open`, `doc.body`, `doc.body_close`

- `site.title`, `site.base_url`, `site.logo`, `site.favicon`
- `site.sidebar`, `site.sidebar_sections`, `site.toc`, `site.menus`
- `styles`, `scripts`, `syntax_css`, `theme`, `target`

Navigation entries also expose `href`, `label`, `label_html`, and `active`.

Here is a minimal `layouts/notebook.html`:

```
{{ doc.head }}
  <meta charset="UTF-8">
  <title>{{ doc.title }}</title>
  {% for style in styles %}
  <style>
{{ style.css }}
  </style>
  {% endfor %}
{{ doc.body_open }}
  <header class="document-header">
    <a href="index.html">Home</a>
    <button type="button" data-calepin-theme-toggle>Theme</button>
  </header>
  <main class="container">
    {{ doc.body }}
  </main>
  {% for script in scripts %}
  <script>
{{ script.content }}
  </script>
  {% endfor %}
{{ doc.body_close }}
```

Keep `doc.head`, `doc.body_open`, and `doc.body_close` unless you are intentionally replacing the entire HTML shell.

Website layouts

You can switch layouts per page with `<website-metadata>`:

```
#metadata((
  title: "Landing page",
  layout: "layouts/landing.html",
)) <website-metadata>
```

The `layout` value must be a relative `.html` path inside the active theme. Calepin does not add `layouts/` or `.html` for you and does not fall back to `layouts/webpage.html` if the file is missing.

Partials

Keep repeated HTML in partials under `partials/` and include them from layouts.

```
{% include "partials/header.html" %}
```

Partials receive the same template context as the file that includes them.

Shared imports

`theme.toml` can request shared partials, CSS, and JS so a theme uses common pieces from the built-in stack.

```
[shared]
partials = ["site-meta.html", "theme-init.html", "styles.html", "scripts.html", "pagefind-modal.html", "theme-toggle.html"]
css = ["theme.css", "code.css", "widgets.css"]
js = ["theme-toggle.js", "language-picker.js", "copy-code.js"]
```

Shared items load first, then local files in `partials/`, `css/`, and `js/` override by filename if they exist.

Use filenames only (`theme.css`, not `css/theme.css`, and not `../theme.css`).

Notebook Typst templates

`notebook.typ.jinja` is the Typst-side wrapper used by notebook outputs.

```
themes/my-theme/
  notebook.typ.jinja
```

Before Typst runs, Calepin renders this file with MiniJinja so the output is still valid Typst source.

To customize it:

1. copy a local theme
2. edit `themes/my-theme/notebook.typ.jinja`
3. set `theme = "themes/my-theme"` in `calepin.toml`

Inside the template, place notebook content with `document.body`:

```
#set page(
  paper: "us-letter",
  margin: (x: 1in, y: 0.85in),
  numbering: "1",
)

#set text(font: "Libertinus Serif", size: 10.5pt)

{{ document.body }}
```

Useful `notebook.typ.jinja` values:

- `theme`: local theme directory name
- `target`: `notebook`
- `document.path`: `.typ` input path relative to workspace
- `document.dir`: input directory relative to workspace
- `document.stem`: input filename without `.typ`
- `document.body`: notebook body, injected as a `#include`
- `document.meta`: values from `#metadata(...) <website-metadata>`
- `params`: CLI parameter map

If `document.body` is not referenced, Calepin appends the notebook body after the rendered template.

`theme = "typst"` disables notebook-specific theming, or use an empty `notebook.typ.jinja` for a minimal pass-through template. Older themes using `paged.typ.jinja` still work; new themes should use `notebook.typ.jinja`.